

Deep Learning

Back propagation(Bp)



Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir





https://www.aparat.com/mehran.safayani

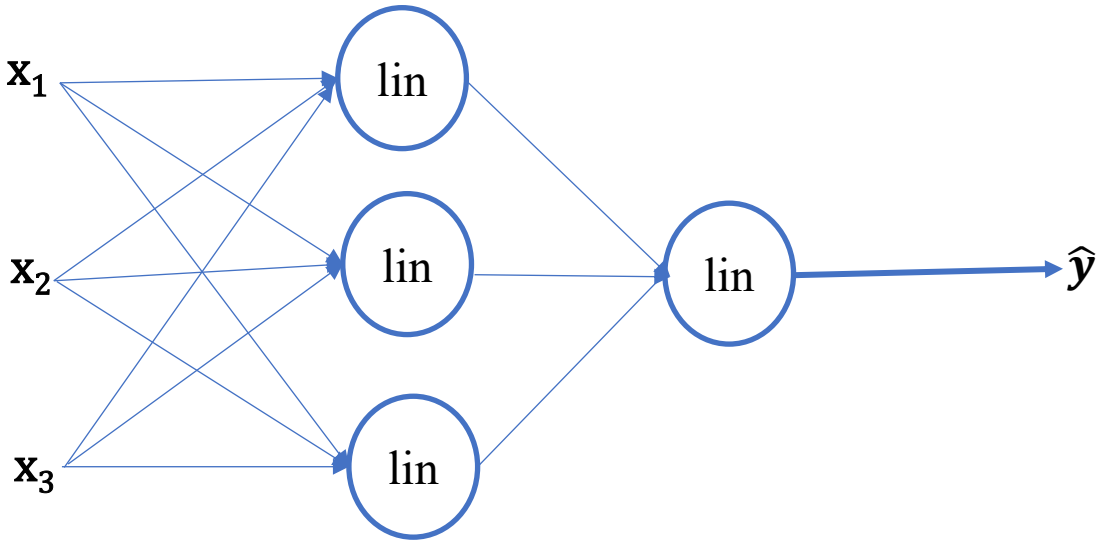


https://github.com/safayani/deep_learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Activation function

Non-linear activation function



- چرا به توابع فعال‌ساز غیرخطی نیاز داریم؟

- با فرض تابع فعال‌ساز خطی

- $z^{[1]} = w^{[1]}x + b^{[1]}$
- $a^{[1]} = g(z^{[1]}) = z^{[1]}$
- $z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$
- $a^{[2]} = g(z^2) = z^{[2]}$

$$a^{[1]} = z^{[1]} = w^{[1]}x + b^{[1]}$$
$$a^{[2]} = z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$
$$= w^{[2]}(w^{[1]}x + b^{[1]}) + b^{[2]}$$
$$= \underbrace{w^{[2]}w^{[1]}}_{w'}x + \underbrace{w^{[2]}b^{[1]} + b^{[2]}}_{b'}$$
$$= w'x + b'$$

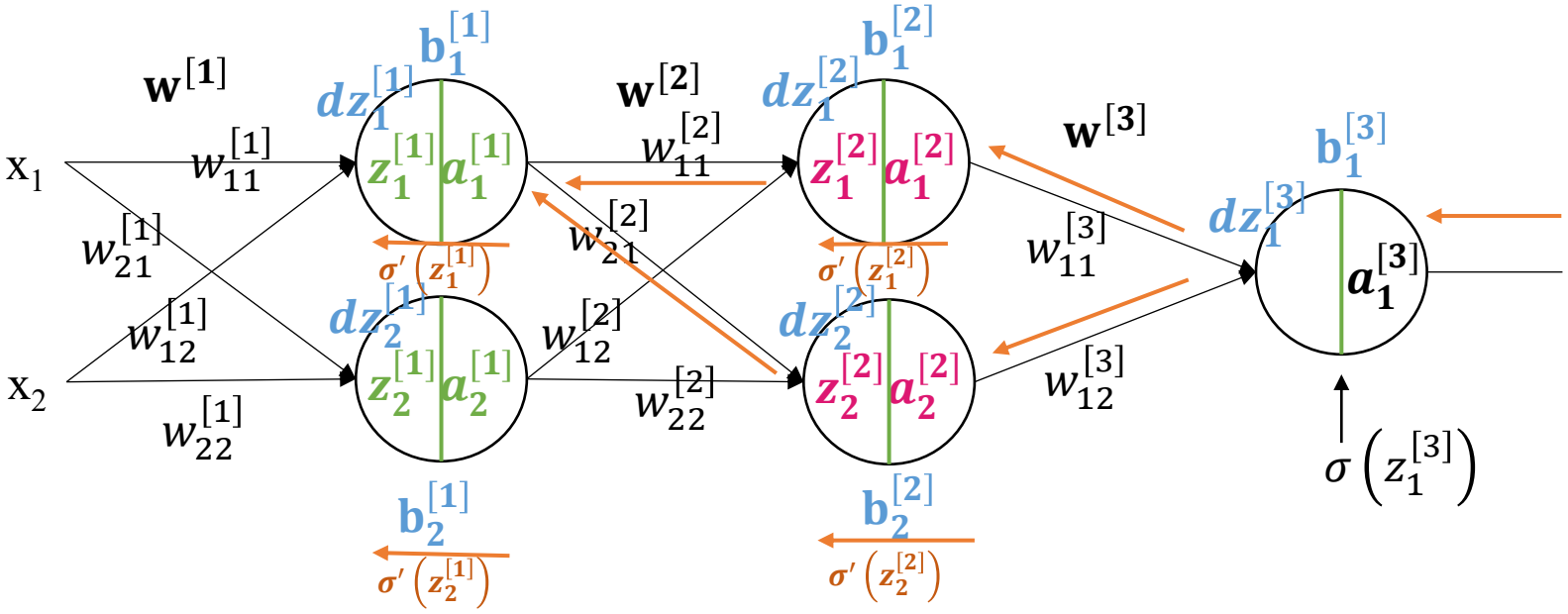
Derivatives of activation functions

- Sigmoid:
- $g(z) = \frac{1}{1+e^{-z}} \qquad g'(z) = \frac{dg(z)}{dz} = \frac{1}{1+e^{-z}}\left(1 - \frac{1}{1+e^{-z}}\right) = g(z)(1 - g(z))$
- Tanh:
- $g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \qquad g'(z) = 1 - \tanh^2(z) = 1 - g^2(z)$
- Relu:
- $g(z) = \max(0, z) = \begin{cases} z & z \geq 0 \\ 0 & z < 0 \end{cases} \qquad g'(z) = \begin{cases} 1 & if\ z \geq 0 \\ 0 & if\ z < 0 \end{cases}$
- Leaky Relu:
- $g(z) = \max(z, 0.01z) = \begin{cases} z & z \geq 0 \\ 0.01z & z < 0 \end{cases} \qquad g'(z) = \begin{cases} 1 & if\ z \geq 0 \\ 0.01 & if\ z < 0 \end{cases}$





Back propagation(Bp)



- $dw_{11}^{[2]} = \frac{L(\ldots, w_{11}^{[2]}+\varepsilon, \ldots) - L(\ldots, w_{11}^{[2]}-\varepsilon, \ldots)}{2\varepsilon}$

- پس‌انتشار

$\mathbf{w_{ij}^{[l]}}$

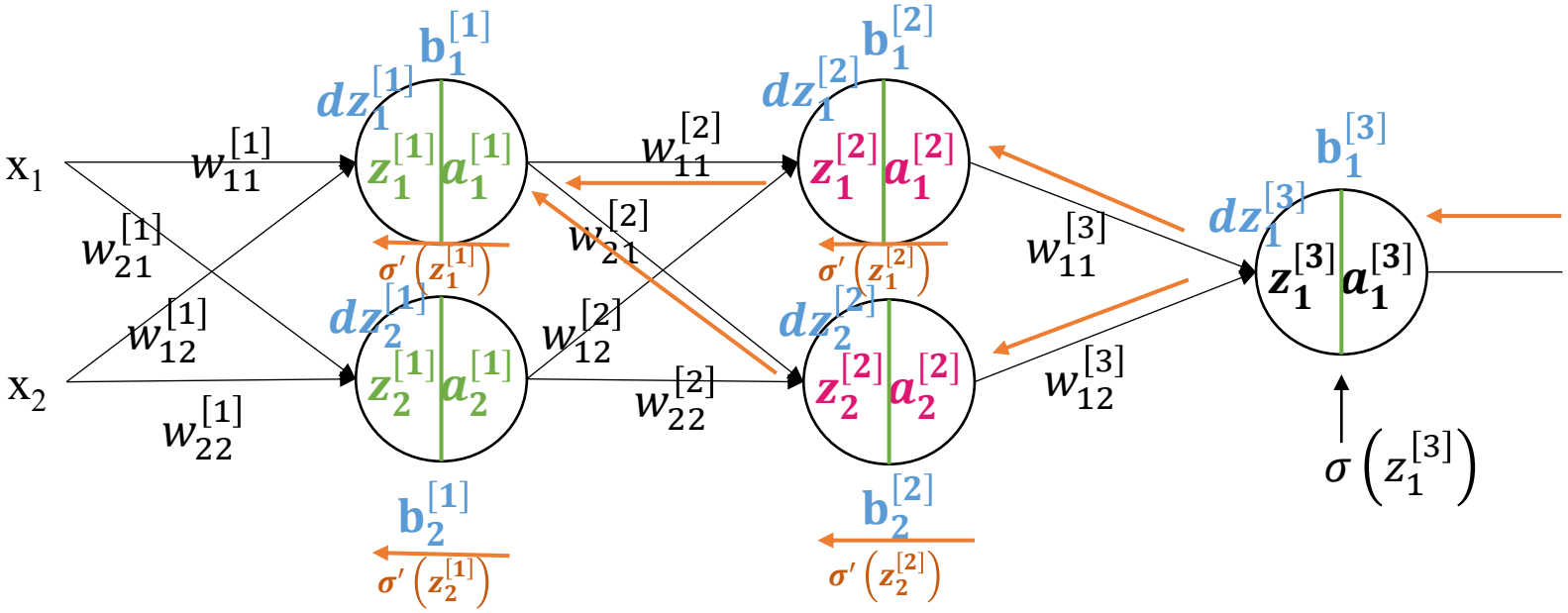
$$\frac{dL}{dw_{ij}^{l}} \quad \text{for } l = 1 \ldots L$$

for each i,j

$$\mathbf{w_{ij}^{[l]}} = \mathbf{w_{ij}^{[l]}} - \alpha \frac{dL}{dw_{ij}^{l}}$$

Back propagation(Bp)

- پس‌انتشار



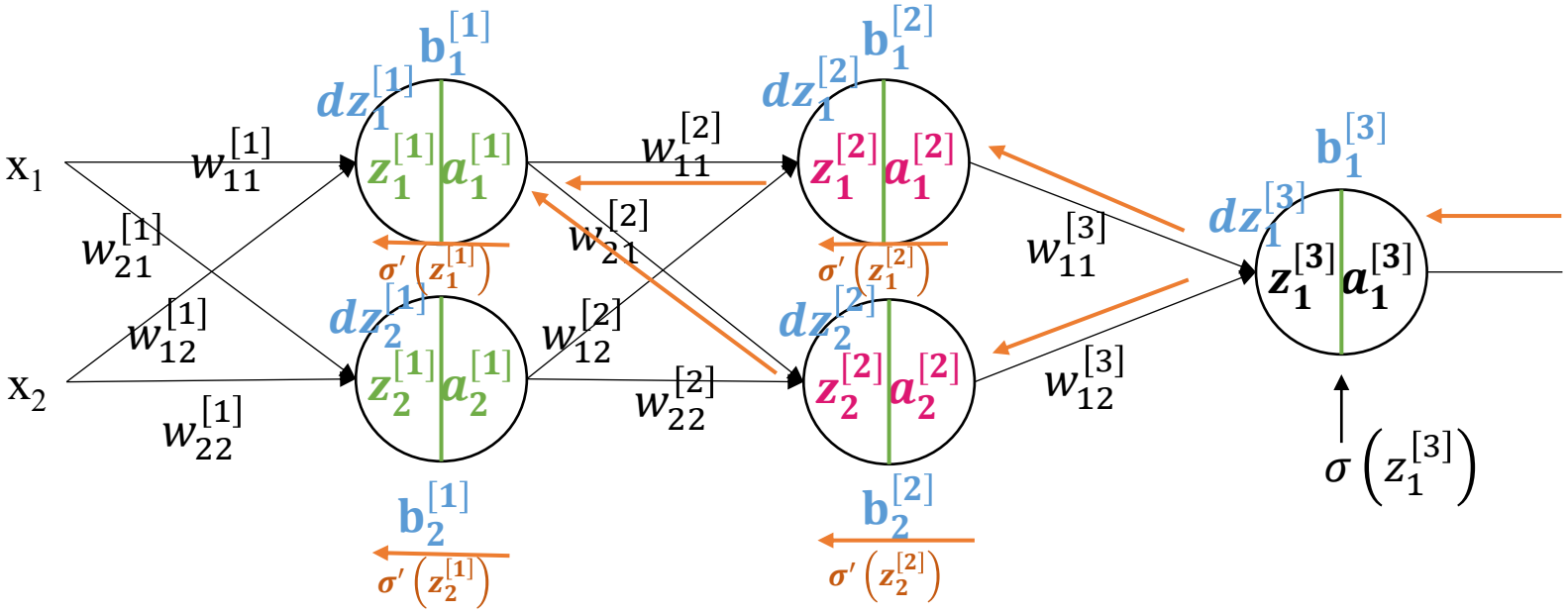
$$\frac{dL}{dw_{11}^{[3]}} = \underbrace{\frac{dL}{da_1^{[3]}} \times \frac{da_1^{[3]}}{dz_1^{[3]}}}_{\frac{dL}{dz_1^{[3]}} = dz_1^{[3]}} \times \frac{dz_1^{[3]}}{dw_{11}^{[3]}} = \left(a_1^{[3]} - y\right) a_1^{[2]}$$

$$z_1^{[3]} = a_1^{[2]} w_{11}^{[3]} + a_2^{[2]} w_{12}^{[3]} + b_1^{[3]}$$

$$\frac{dL}{db_1^{[3]}} = \frac{dL}{dz_1^{[3]}} \times \underbrace{\frac{dz_1^{[3]}}{db_1^{[3]}}}_{1} = dz_1^{[3]} \times 1 = dz_1^{[3]}$$

Back propagation(Bp)

- پس‌انتشار



$$dz_1^{[2]} = \frac{dL}{dz_1^{[2]}} = \underbrace{\frac{dL}{dz_1^{[3]}}}_{dz_1^{[3]}} \times \underbrace{\frac{dz_1^{[3]}}{da_1^{[2]}}}_{w_{11}^{[3]}} \times \underbrace{\frac{da_1^{[2]}}{dz_1^{[2]}}}_{\sigma'(z_1^{[2]})} \qquad a_1^{[2]} = \sigma\left(z_1^{[2]}\right) \qquad \frac{da_1^{[2]}}{dz_1^{[2]}} = \sigma'\left(z_1^{[2]}\right)$$

$$dz_1^{[2]} = dz_1^{[3]} \times w_{11}^{[3]} \times \sigma'\left(z_1^{[2]}\right)$$

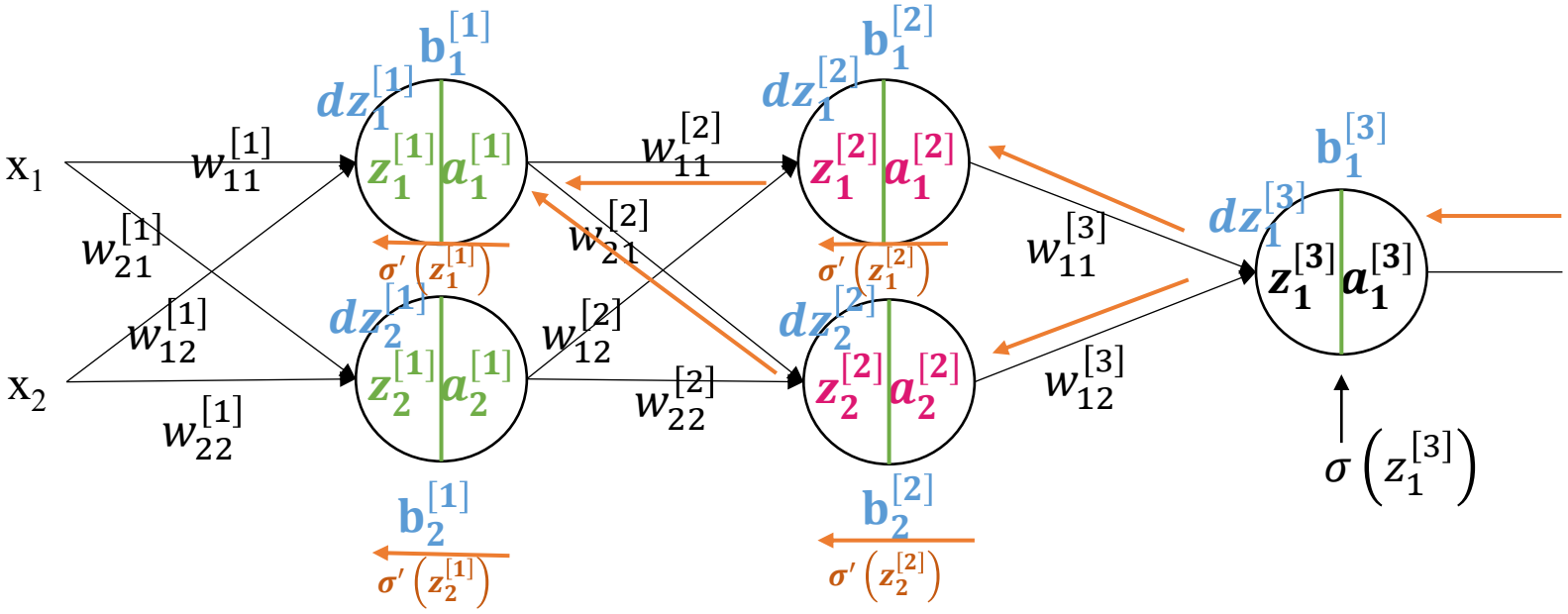
$$dz_2^{[2]} = dz_1^{[3]} \times w_{12}^{[3]} \times \sigma'\left(z_2^{[2]}\right)$$

$$dw_{11}^{[2]} = dz_1^{[2]} \times a_1^{[1]} \qquad db_1^{[2]} = dz_1^{[2]}$$

$$dw_{12}^{[2]} = dz_1^{[2]} \times a_2^{[1]} \qquad db_2^{[2]} = dz_2^{[2]}$$

Back propagation(Bp)

- پس‌انتشار



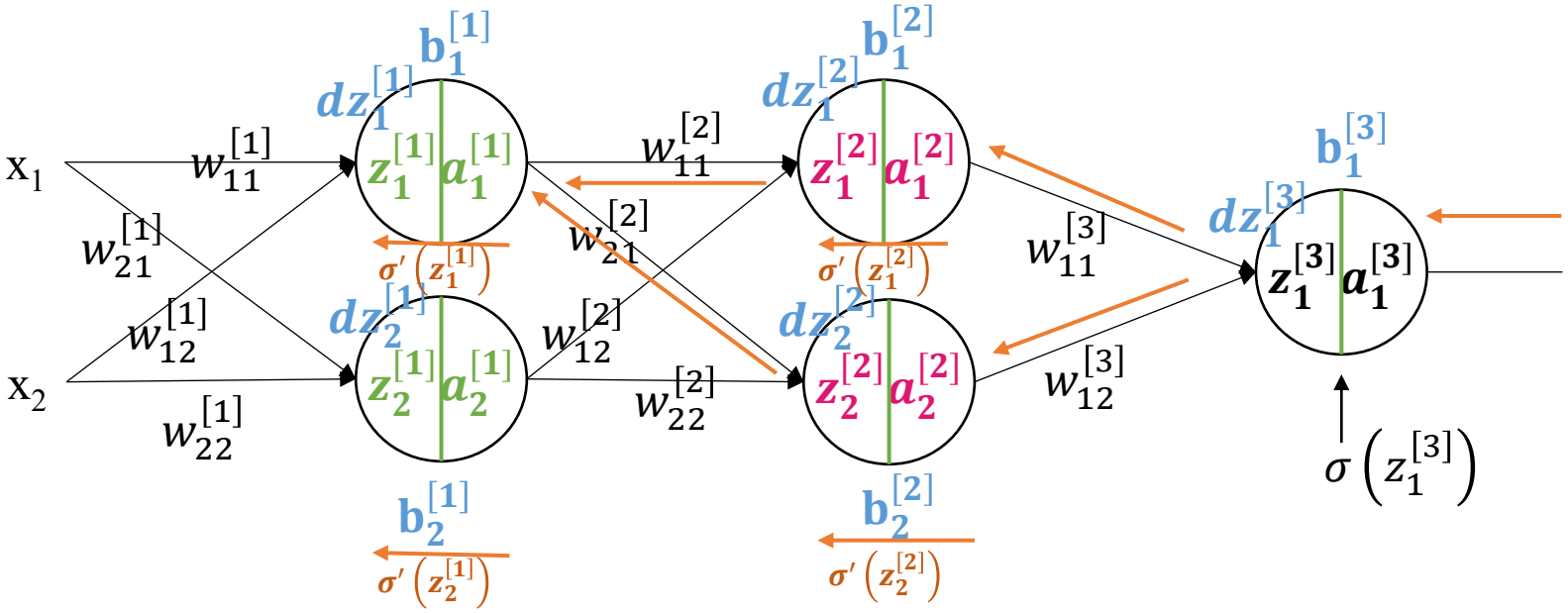
$$dz_1^{[1]} = \frac{dL}{dz_1^{[1]}} = \underbrace{\frac{dL}{dz_1^{[2]}}}_{dz_1^{[2]}} \times \underbrace{\frac{dz_1^{[2]}}{da_1^{[1]}}}_{w_{11}^{[2]}} \times \underbrace{\frac{da_1^{[1]}}{dz_1^{[1]}}}_{\sigma'(z_1^{[1]})} + \underbrace{\frac{dL}{dz_2^{[2]}}}_{dz_2^{[2]}} \times \underbrace{\frac{dz_2^{[2]}}{da_1^{[1]}}}_{w_{21}^{[2]}} \times \underbrace{\frac{da_1^{[1]}}{dz_1^{[1]}}}_{\sigma'(z_1^{[1]})}$$

$$dz_1^{[1]} = (dz_1^{[2]} w_{11}^{[2]} + dz_2^{[2]} w_{21}^{[2]})\, \sigma'\left(z_1^{[1]}\right) \qquad db_1^{[1]} = dz_1^{[1]}$$

$$dz_2^{[1]} = (dz_1^{[2]} w_{12}^{[2]} + dz_2^{[2]} w_{22}^{[2]})\, \sigma'\left(z_2^{[1]}\right) \qquad db_2^{[1]} = dz_2^{[1]}$$

Back propagation(Bp)

- پس‌انتشار



$$dw_{11}^{[1]} = dz_1^{[1]} \times x_1$$

$$dw_{21}^{[1]} = dz_2^{[1]} \times x_1$$

$$dw_{12}^{[1]} = dz_1^{[1]} \times x_2$$

$$dw_{22}^{[1]} = dz_2^{[1]} \times x_2$$

Gradient descent

Gradient descent

- Parameters: $\underbrace{w^{[1]}}_{(n^{[1]},n^{[0]})}$, $\underbrace{b^{[1]}}_{(n^{[1]},1)}$, $\underbrace{w^{[2]}}_{(n^{[2]},n^{[1]})}$, $\underbrace{b^{[2]}}_{(n^{[2]},1)}$
- $w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]} \leftarrow$ random initialization
- **Repeat{**

  →Forward propagation to compute $\hat{y}^{(i)}$ i=1, …, m

  Backward prop

  $\rightarrow dw^{[1]} = \frac{dJ}{dw^{[1]}}, db^{[1]}, dw^{[2]}, db^{[2]}$

  Update
  $$\begin{cases} w^{[1]} = w^{[1]} - \alpha\, dw^{[1]} \\ w^{[2]} = w^{[2]} - \alpha\, dw^{[2]} \\ b^{[1]} = b^{[1]} - \alpha\, db^{[1]} \\ b^{[2]} = b^{[2]} - \alpha\, db^{[2]} \end{cases}$$

  }Until Convergence





Gradient descent

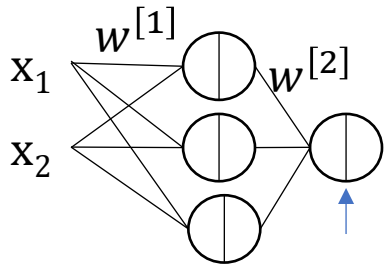
- *Forward propagation:*

$$z^{[1]} = w^{[1]}X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = w^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]})$$



- *Back prop. :*

$$dz^{[2]} = A^{[2]}{}_{1\times m} - Y_{1\times m}$$

$$\underbrace{dw^{[2]}}_{1\times n^{[1]}} = \frac{1}{m}\underbrace{dz^{[2]}}_{1\times m}\underbrace{A^{[1]T}}_{m\times n^{[1]}}$$

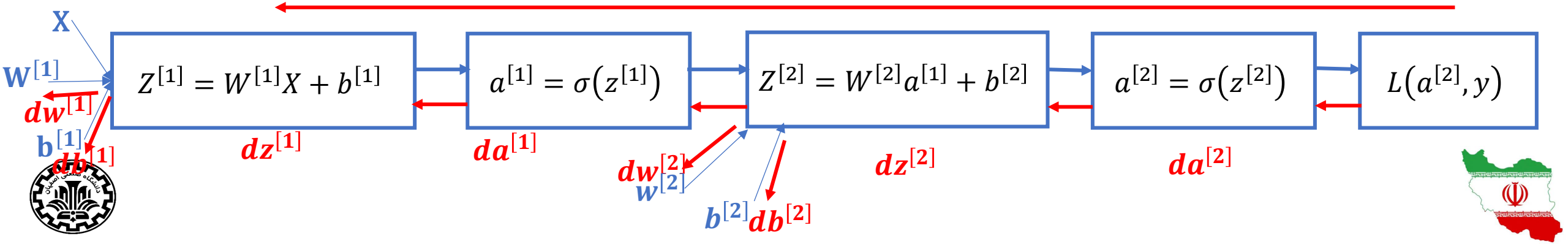
$$\underbrace{db^{[2]}}_{1\times 1} = \frac{1}{m}np\cdot sum\left(\underbrace{dz^{[2]}}_{1\times m}, axis = 1\right)_{(1,1)}$$

Gradient descent

- $\underbrace{dz^{[1]}}_{n^{[1]}\times m} = \underbrace{w^{[2]T}}_{n^{[1]}\times 1}\underbrace{dz^{[2]}}_{1\times m} * \underbrace{g^{[1]'}(\underbrace{z^{[1]}}_{n^{[1]}\times m})}_{n^{[1]}\times m}$

Element wise

- $\underbrace{dw^{[1]}}_{n^{[1]}\times n^{[0]}} = \frac{1}{m}\underbrace{dz^{[1]}}_{n^{[1]}\times m}\underbrace{X^T}_{m\times n^{[0]}}$
- $\underbrace{db^{[1]}}_{n^{[1]}\times 1} = \frac{1}{m} np\cdot sum\left(\underbrace{dz^{[1]}}_{n^{[1]}\times m}, axis = 1, keepdims = True\right)^{(n^{[1]},1)}$



Gradient descent

- $dz^{[1]} = w^{[2]T}\, dz^{[2]} * g^{[1]'}(z^{[1]})$ m=1
- $dz^{[2]} = a^{[2]} - y$
- $dw^{[1]} = dz^{[1]} X^T$
- $dw^{[2]} = dz^{[2]} a^{[1]^T}$
- $db^{[1]} = dz^{[1]}$
- $db^{[2]} = dz^{[2]}$





Summary of gradient descent

- $dz^{[2]} = A^{[2]} - Y$
- $dw^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$
- $db^{[2]} = \frac{1}{m} np \cdot sum\left(dz^{[2]}, axis = 1, keepdims = True\right)$
- $dz^{[1]} = w^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$
- $dw^{[1]} = \frac{1}{m} dz^{[1]} X^{T}$
- $db^{[1]} = \frac{1}{m} np \cdot sum\left(dz^{[1]}, axis = 1, keepdims = True\right)$









Deep Learning

Computational and Memory Complexity of Multilayer Perceptrons (MLPs)

Dr. Mehran Safayani

safayani@iut.ac.ir

safayani.iut.ac.ir





https://www.aparat.com/mehran.safayani



https://github.com/safayani/deep_learning_course

Department of Electrical and computer engineering, Isfahan university of technology, Isfahan, Iran

Computational Complexity: Forward & Backward Propagation

Step	Operation	Dimension Formula	Time Complexity ($\mathcal{O}$)
Forward	$Z^{[1]} = W^{[1]}X + b^{[1]}$	$\left(n^{[1]} \times n^{[0]}\right) \cdot \left(n^{[0]} \times m\right)$	$\mathcal{O}\left(m \cdot n^{[0]}n^{[1]}\right)$
	$A^{[1]} = g^{[1]}\left(Z^{[1]}\right)$	Element-wise activation on $\left(n^{[1]} \times m\right)$	$\mathcal{O}\left(m \cdot n^{[1]}\right)$
	$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$	$\left(n^{[2]} \times n^{[1]}\right) \cdot \left(n^{[1]} \times m\right)$	$\mathcal{O}\left(m \cdot n^{[1]}n^{[2]}\right)$
	$A^{[2]} = g^{[2]}\left(Z^{[2]}\right)$	Element-wise activation on $\left(n^{[2]} \times m\right)$	$\mathcal{O}\left(m \cdot n^{[2]}\right)$
Backward	$dZ^{[2]} = A^{[2]} - Y$	Element-wise subtraction	$\mathcal{O}\left(m \cdot n^{[2]}\right)$
	$dW^{[2]} = \frac{1}{m} dZ^{[2]}\left(A^{[1]}\right)^T$	$\left(n^{[2]} \times m\right) \cdot \left(m \times n^{[1]}\right)$	$\mathcal{O}\left(m \cdot n^{[1]}n^{[2]}\right)$
	$db^{[2]} = \frac{1}{m}\sum d\,Z^{[2]}$	Row-wise reduction sum	$\mathcal{O}\left(m \cdot n^{[2]}\right)$
	$dZ^{[1]} = \left(W^{[2]}\right)^T dZ^{[2]} * g^{[1]'}\left(Z^{[1]}\right)$	$\left(n^{[1]} \times n^{[2]}\right) \cdot \left(n^{[2]} \times m\right)$ + element−wise	$\mathcal{O}\left(m \cdot n^{[1]}n^{[2]}\right)$
	$dW^{[1]} = \frac{1}{m} dZ^{[1]}X^T$	$\left(n^{[1]} \times m\right) \cdot \left(m \times n^{[0]}\right)$	$\mathcal{O}\left(m \cdot n^{[0]}n^{[1]}\right)$
	$db^{[1]} = \frac{1}{m}\sum d\,Z^{[1]}$	Row-wise reduction sum	$\mathcal{O}\left(m \cdot n^{[1]}\right)$

Overall Order of the Algorithm

- **Dominant Operations:** Matrix multiplications dominate all element-wise additions, activations, and reduction sums.
- **Per Iteration Cost:**
- $\mathcal{O}\left(m \cdot \left(n^{[0]} n^{[1]} + n^{[1]} n^{[2]}\right)\right)$
- *(Since* $n^{[2]} = 1$*, this simplifies to* $\mathcal{O}\left(m \cdot n^{[1]}\left(n^{[0]} + 1\right)\right) \approx \mathcal{O}\left(m \cdot n^{[0]} n^{[1]}\right)$*)*
- **General Rule for an L-layer Network:**
- Time Complexity per Epoch $= \mathcal{O}\left(m \sum_{l=1}^{L} n^{[l-1]}\, n^{[l]}\right) = \mathcal{O}(m \cdot |\, \mathcal{W}\, |)$
- where $|\, \mathcal{W}\, |$is the total number of network parameters (weights)

Key Takeaways

- **Backward ≈Forward:** Backpropagation takes approximately $\mathbf{2}$ ×the compute of Forward propagation due to computing gradients both with respect to weights (dW) and activations (dZ).

- **Forward Pass:** $\approx 2 \cdot m \cdot |\mathcal{W}|$ operations (1 multiply + 1 add per parameter $\times m$)
- **Backward Pass:** $\approx 4 \cdot m \cdot |\mathcal{W}|$ operations (two separate matrix multiplications)
- **Total per iteration:** $\approx 6 \cdot m \cdot |\mathcal{W}|$ FLOPs





Static Memory (Independent of batch size m)

Component	Matrices / Tensors	Exact Number of Floats
Weights (W)	$W^{[1]} \in \mathbb{R}^{n^{[1]} \times n^{[0]}}, W^{[2]} \in \mathbb{R}^{n^{[2]} \times n^{[1]}}$	$n^{[1]}n^{[0]} + n^{[2]}n^{[1]}$
Biases (b)	$b^{[1]} \in \mathbb{R}^{n^{[1]} \times 1}, b^{[2]} \in \mathbb{R}^{n^{[2]} \times 1}$	$n^{[1]} + n^{[2]}$
Weight Gradients (dW)	$dW^{[1]}, dW^{[2]}$	$n^{[1]}n^{[0]} + n^{[2]}n^{[1]}$
Bias Gradients (db)	$db^{[1]}, db^{[2]}$	$n^{[1]} + n^{[2]}$
Total Static Elements (N_{static})	Parameters + Gradients	$2 \times (n^{[1]}n^{[0]} + n^{[2]}n^{[1]} + n^{[1]} + n^{[2]})$





Dynamic / Activation Memory (Scales with batch size m)

- To compute the backward pass:
- $dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T \implies$ requires X
- $dZ^{[1]} = \left(W^{[2]}\right)^T dZ^{[2]} * g^{[1]'}\left(Z^{[1]}\right) \implies$ requires $Z^{[1]}$(or $A^{[1]}$)
- $dW^{[2]} = \frac{1}{m} dZ^{[2]} \left(A^{[1]}\right)^T \implies$ requires $A^{[1]}$
- $dZ^{[2]} = A^{[2]} - Y \implies$ requires $A^{[2]}$and target Y
- Intermediate error terms generated during backprop: $dZ^{[2]}$and $dZ^{[1]}$





Dynamic / Activation Memory (Scales with batch size m)

Component	Matrices / Tensors	Exact Number of Floats
Input Data (X) & Target (Y)	$X \in \mathbb{R}^{n^{[0]} \times m}, Y \in \mathbb{R}^{n^{[2]} \times m}$	$m \cdot \left(n^{[0]} + n^{[2]}\right)$
Saved Hidden Activations	$Z^{[1]} \in \mathbb{R}^{n^{[1]} \times m}, A^{[1]} \in \mathbb{R}^{n^{[1]} \times m}$	$2m \cdot n^{[1]}$(or $m \cdot n^{[1]}$if ReLU cache can be reused)
Saved Output Activation	$A^{[2]} \in \mathbb{R}^{n^{[2]} \times m}$	$m \cdot n^{[2]}$
Backward Workspaces (dZ)	$dZ^{[1]} \in \mathbb{R}^{n^{[1]} \times m}, dZ^{[2]} \in \mathbb{R}^{n^{[2]} \times m}$	$m \cdot \left(n^{[1]} + n^{[2]}\right)$
Total Dynamic Elements (N_{dynamic})	Activations + Inputs + Backward Buffers	$m \cdot (n^{[0]} + 3n^{[1]} + 3n^{[2]})$





Exact Total Memory Formula

- Total Number of Floating-Point Values (N_{total}):

$$N_{\text{total}} = \underbrace{2\left(n^{[1]}n^{[0]} + n^{[2]}n^{[1]} + n^{[1]} + n^{[2]}\right)}_{\text{Parameters } (\mathcal{W}) + \text{Gradients } (\nabla\mathcal{W})} + \underbrace{m\left(n^{[0]} + 3n^{[1]} + 3n^{[2]}\right)}_{\text{Data, Cached Activations \& Workspaces}}$$

- For an Arbitrary L-Layer MLP (General Form)

$$N_{\text{total}} = \underbrace{2\sum_{l=1}^{L}\left(n^{[l]}n^{[l-1]} + n^{[l]}\right)}_{\text{Parameters } (\mathcal{W}) + \text{Gradients } (\nabla\mathcal{W})} + \underbrace{m\left(n^{[0]} + \sum_{l=1}^{L} 3n^{[l]}\right)}_{\text{Batch Data, Activations \& Gradients}}$$